

채팅 시스템

여동현

목차

- 01 요구사항
- 02 개략적 설계
- 03 상세 설계

요구사항

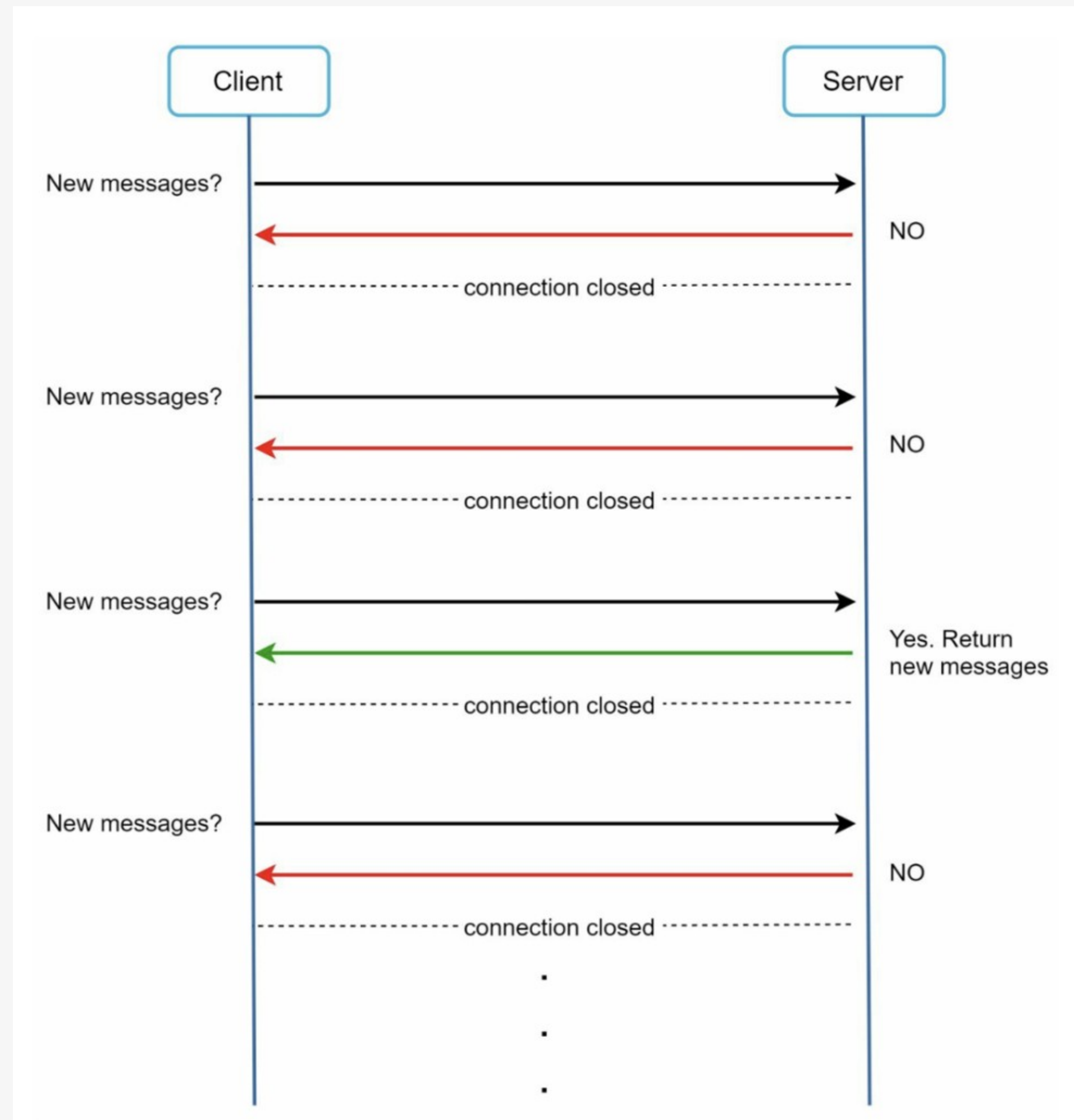
■ 요구사항

- 응답 지연 낮은 일대일 채팅
- 100명 규모 그룹채팅
- 접속 상태 표시 기능
- 1계정 多단말 가능
- 푸시 알림

개략적 설계

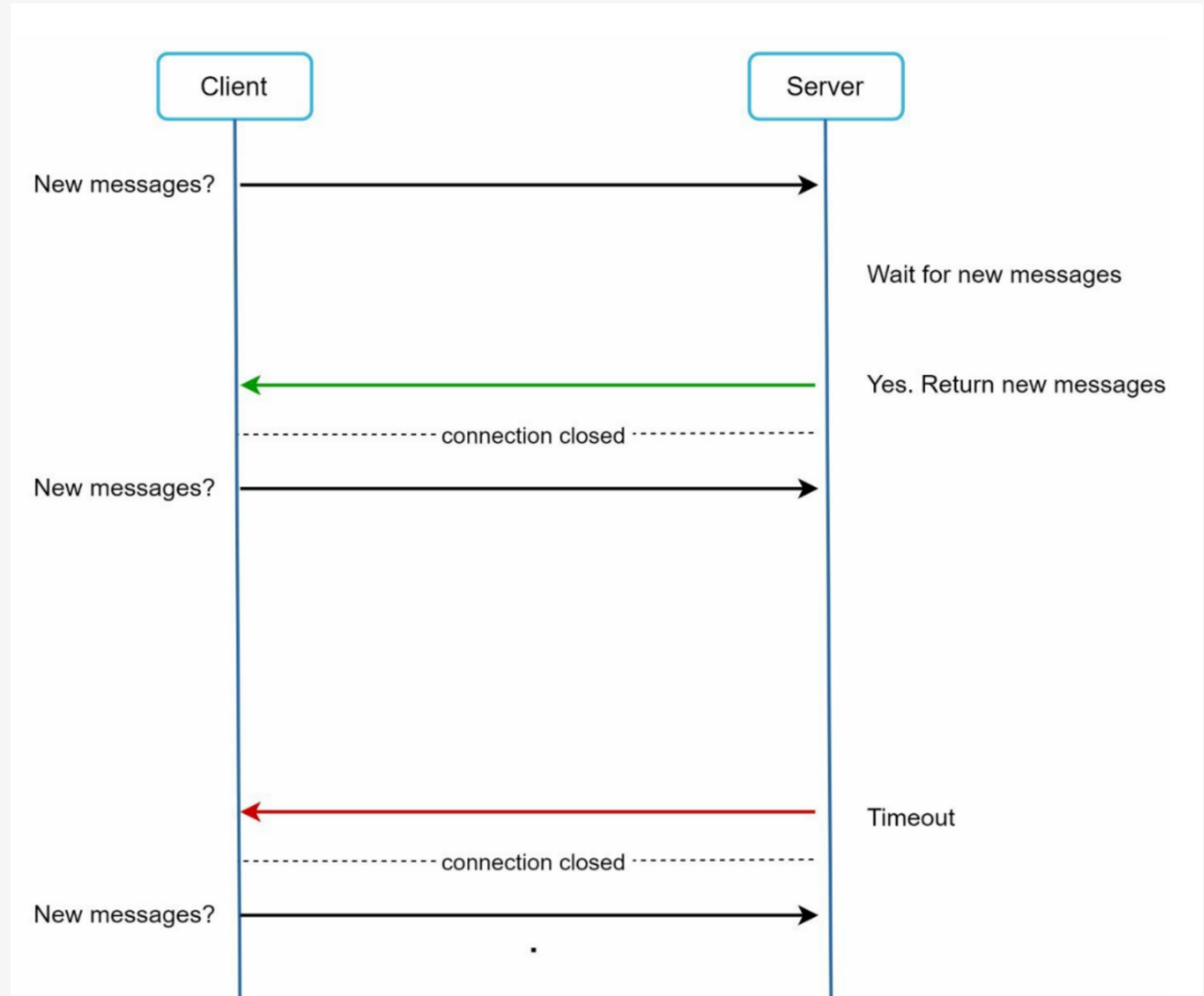
1안 - 폴링

- 딱봐도 비효율적



2안 - 롱 폴링

- 클라 간 서버 불일치
- 서버 기준 클라이언트 연결 해제 인지 불가
- 비효율적



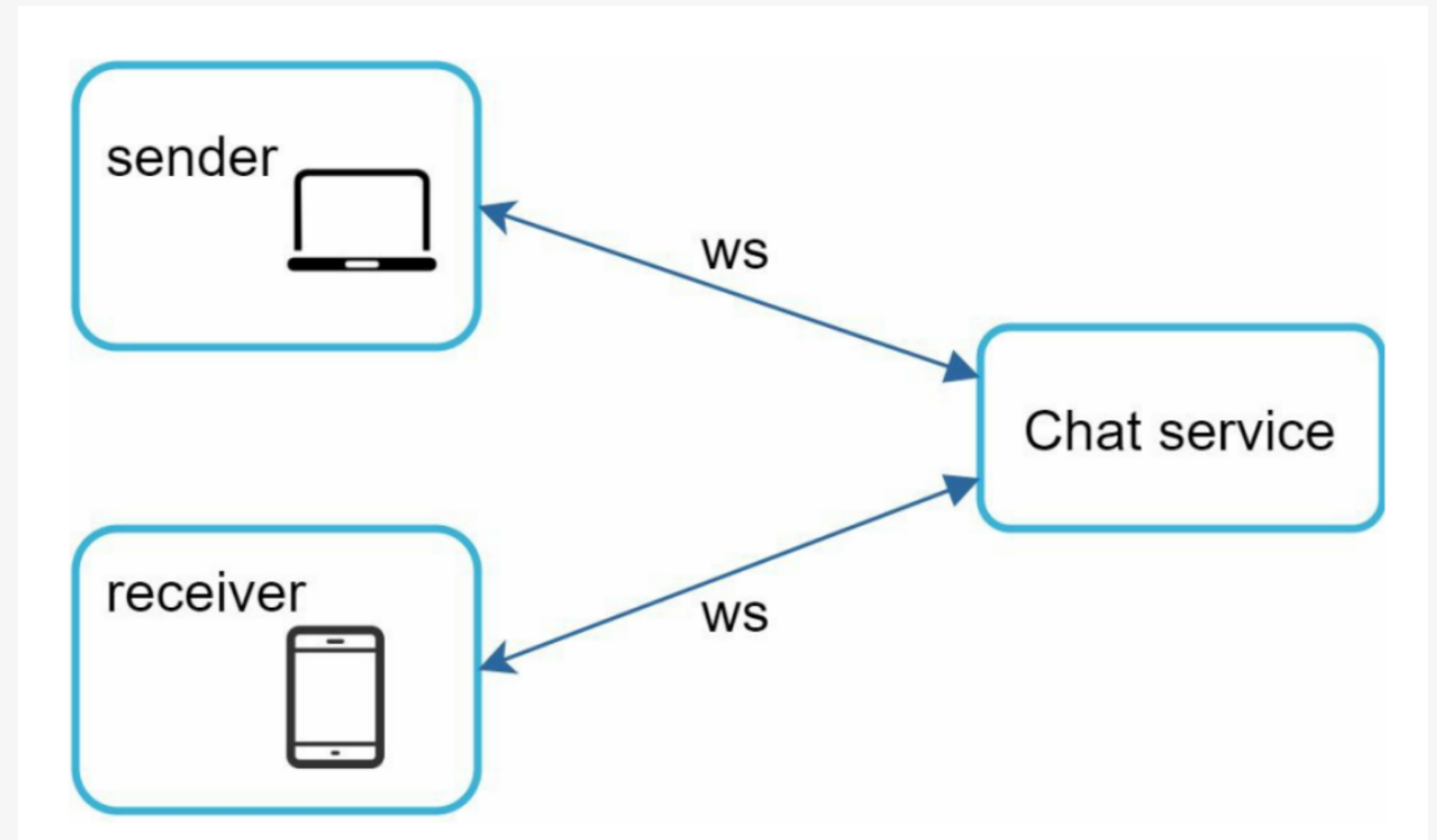
3안 - 웹소켓

1. HTTP로 시작 (3 way-hand shake)
2. 웹소켓으로 변경

특징

- 메시지 송수신 시 동일 프로토콜을 사용
>설계와 구현이 단순, 직관

- 양방향 통신
- 연결 항구 유지



3안 - 웹소켓

HTTP Polling vs WebSocket

실시간 라이브채팅의 특성상 1:1 대화가 아닌 많은 사용자가 동시에 진행하는 채팅이기 때문에 일부 메시지의 순서가 섞이거나 서로 다른 메시지를 보고 있다 하여도 대화의 맥락을 이해하는 데에는 크게 문제는 없다고 생각하여 두 방식 모두 저희의 서비스에 사용은 가능하다는 결론을 내렸고, 그렇다면 우리 서비스의 관점으로 장/단점을 정리해 봤습니다.

Type	장점	단점
HTTP Polling	서비스 구현이 간단하고 편리함	- 유저가 작성한 메시지의 전달 속도가 Polling 주기에 따라 결정됨 - Polling된 시점에 따라 사용자가 보고 있는 채팅의 내용이 상이 할 수 있음 - 메시지의 Sync를 위해 많은 클라이언트에서 Polling 주기를 짧게 조정하면 서버의 부하가 심해짐
Web Socket	- 유저가 작성한 메시지의 전달을 서버에서 진행하므로 즉각 전달이 가능함 - 모든 시점에서 사용자가 보고 있는 채팅의 내용이 일치함	- 구현 방식이 HTTP Polling에 비해 복잡함 - 서버당 연결 가능한 클라이언트 수가 제한됨 - 서버와 Connection을 유지하기 위한 리소스가 낭비됨

■ 개략적 설계 - 무상태 서비스

로그인, 회원가입, 사용자 프로필 표시 등
전통적인 요청 처리.

"서비스 탐색"을 뒤에서 다뤄볼 것

■ 개략적 설계 - 상태 유지 서비스

채팅 서비스가 유일 - 후술

개략적 설계 - 제 3 서비스

푸시 알람 등
3장 참고

■ 개략적 설계 - 규모 확장성

당연하지만 싱글 서버라고 대답하는 것
좋지 않음 (SPOF)

그러나, 시작일 뿐이다. 라고 면접관을 설
득(?)

개략적 설계 - 저장소

데이터 종류

- 일반적인 데이터(친구 목록 등) → 관계형 데이터베이스
- 채팅 이력 → 키-값 저장소
 - 수평적 규모 확장 용이
 - 데이터 접근 지연시간 낮음
 - 이미 많이 쓴다!
- 관계형이 어려운 이유
 - 롱테일 처리가 어렵다!
 - RA 접근이 느려짐

개략적 설계 - 저장소

- 관계형이 어려운 이유
 - 롱테일 처리가 어렵다!
 - RA 접근이 느려짐

~~책 기준 참고 자료: https://en.wikipedia.org/wiki/Long_tail~~
(관련 내용이 없다..?)

1. 캐시에 없음
2. 썬 index 접근이 아니라 B-Tree 접근 (?)
(틀린 내용일 수 있습니다.)

개략적 설계 - 데이터 모델

1:1 용 데이터 데이터 테이블

message	
message_id	bigint
message_from	bigint
message_to	bigint
content	text
created_at	timestamp

개략적 설계 - 데이터 모델

그룹채팅용 데이터 테이블

group_message	
channel_id	bigint
message_id	bigint
user_id	bigint
content	text
created_at	timestamp

채널 id는 아마 디코 채널 같은 거...

개략적 설계 - 데이터 모델

message_id

1. 고유해야함
2. 시간 순서와 일치해야함 (정렬가능)

방법

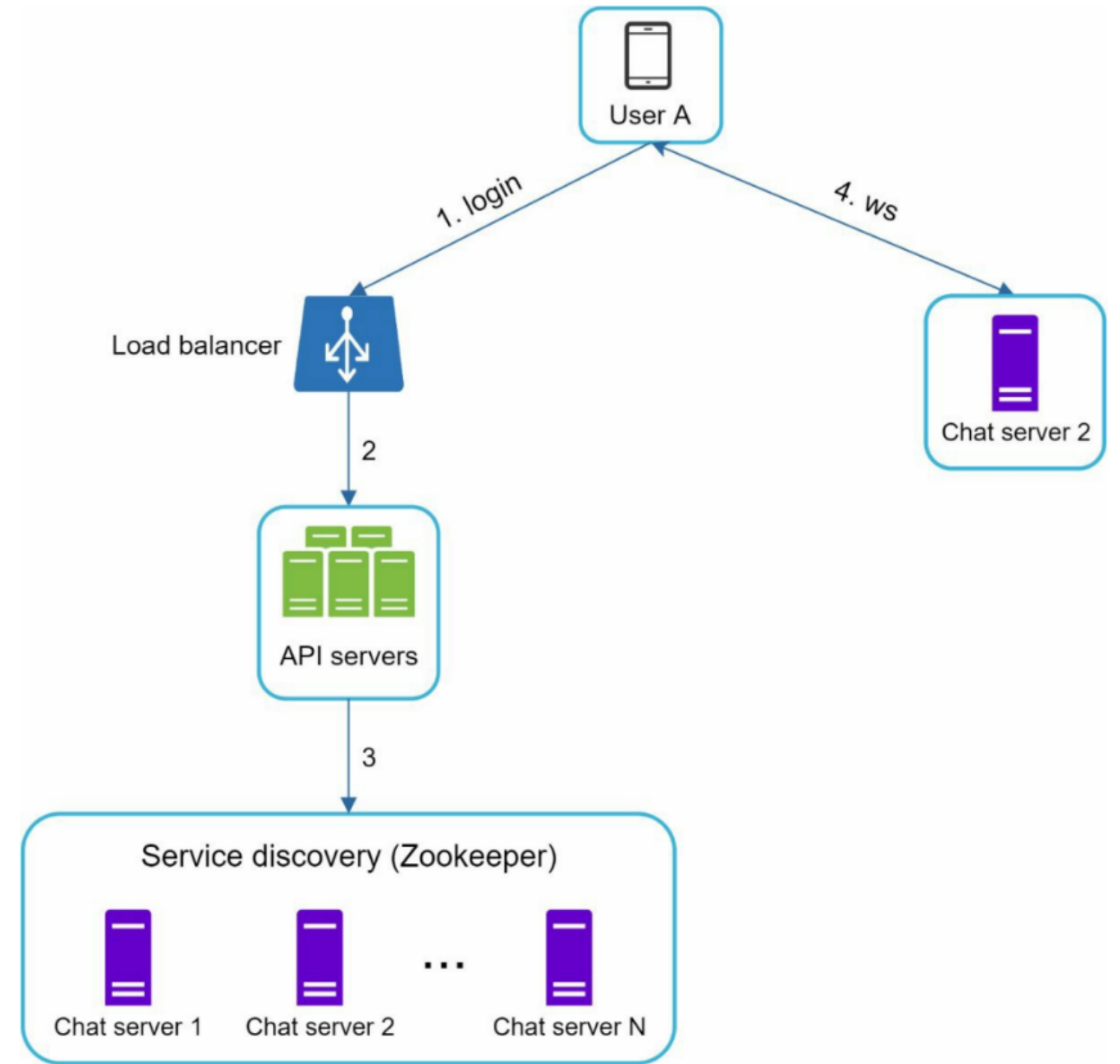
- 1) 전에 배운 유일 Id 생성기 (snowflake 등)
 - 2) 지역적 순서 번호 생성기
- 다른 채널끼리 순서 합칠 일 없으니까

상세 설계

상세 설계 - 서비스 탐색

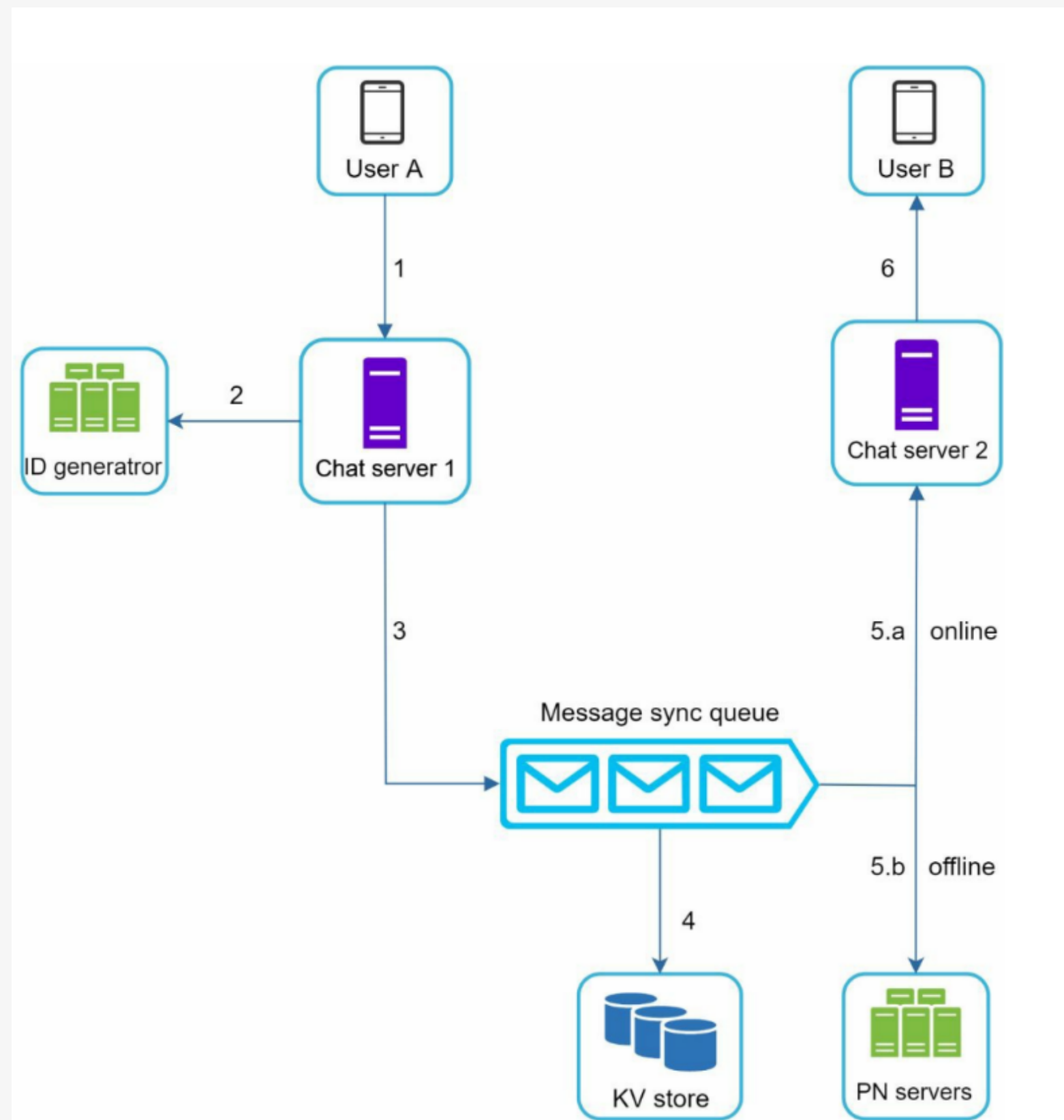
서비스 탐색 == 채팅 서버 추천

대표 서비스: Apache Zookeeper



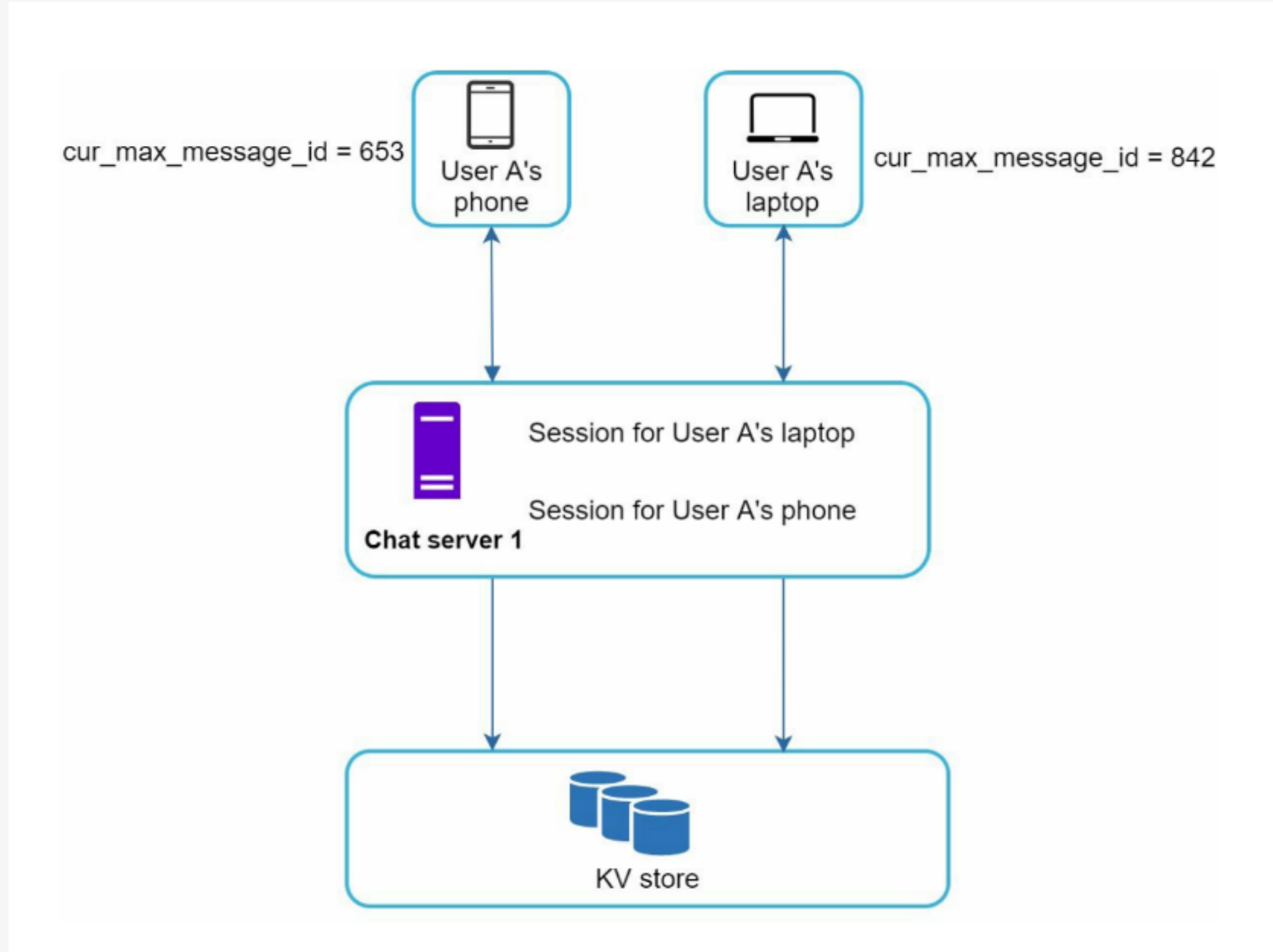
상세 설계 - 메시지 흐름

1:1 채팅



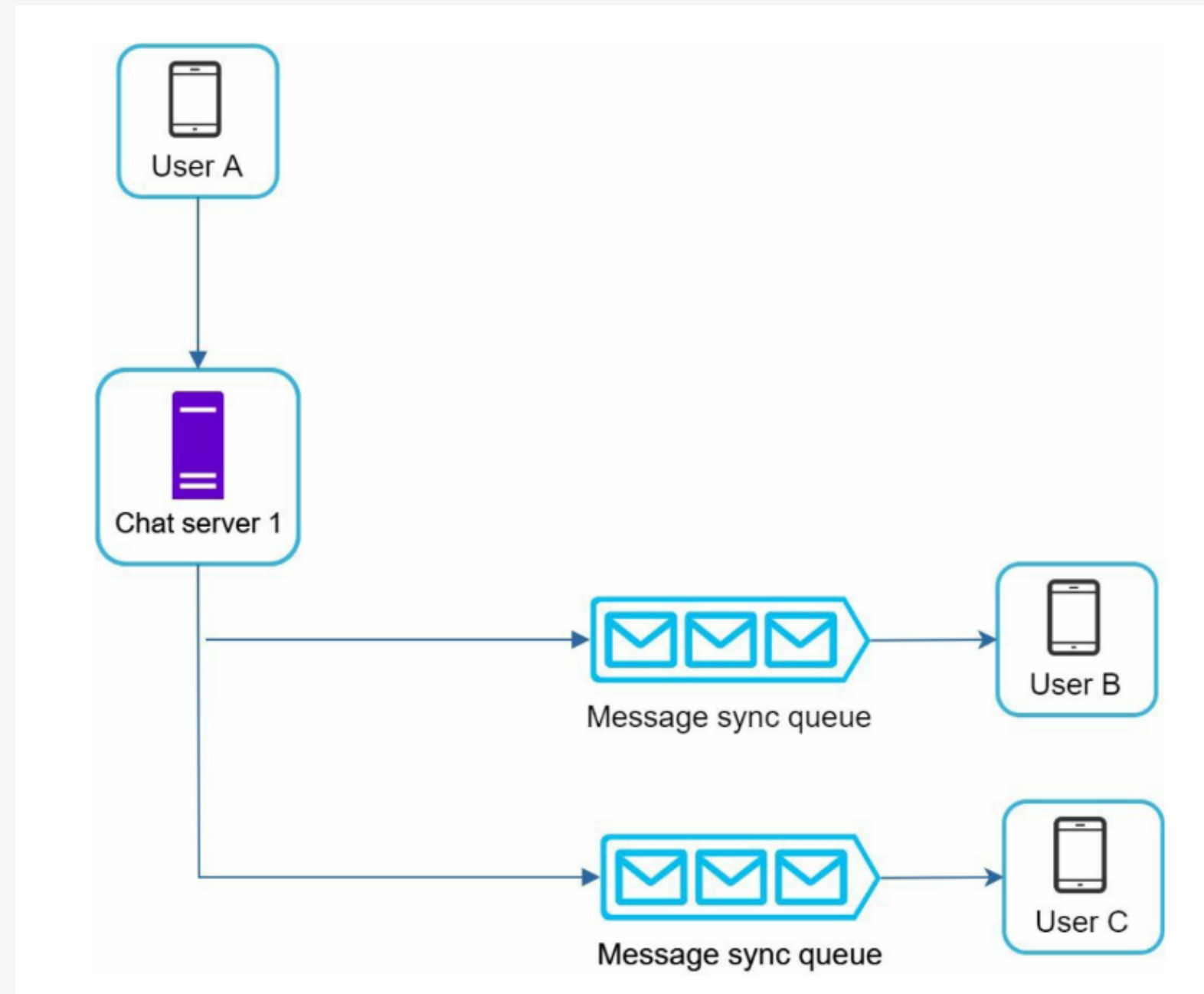
상세 설계 - 메시지 흐름

기기 동기화



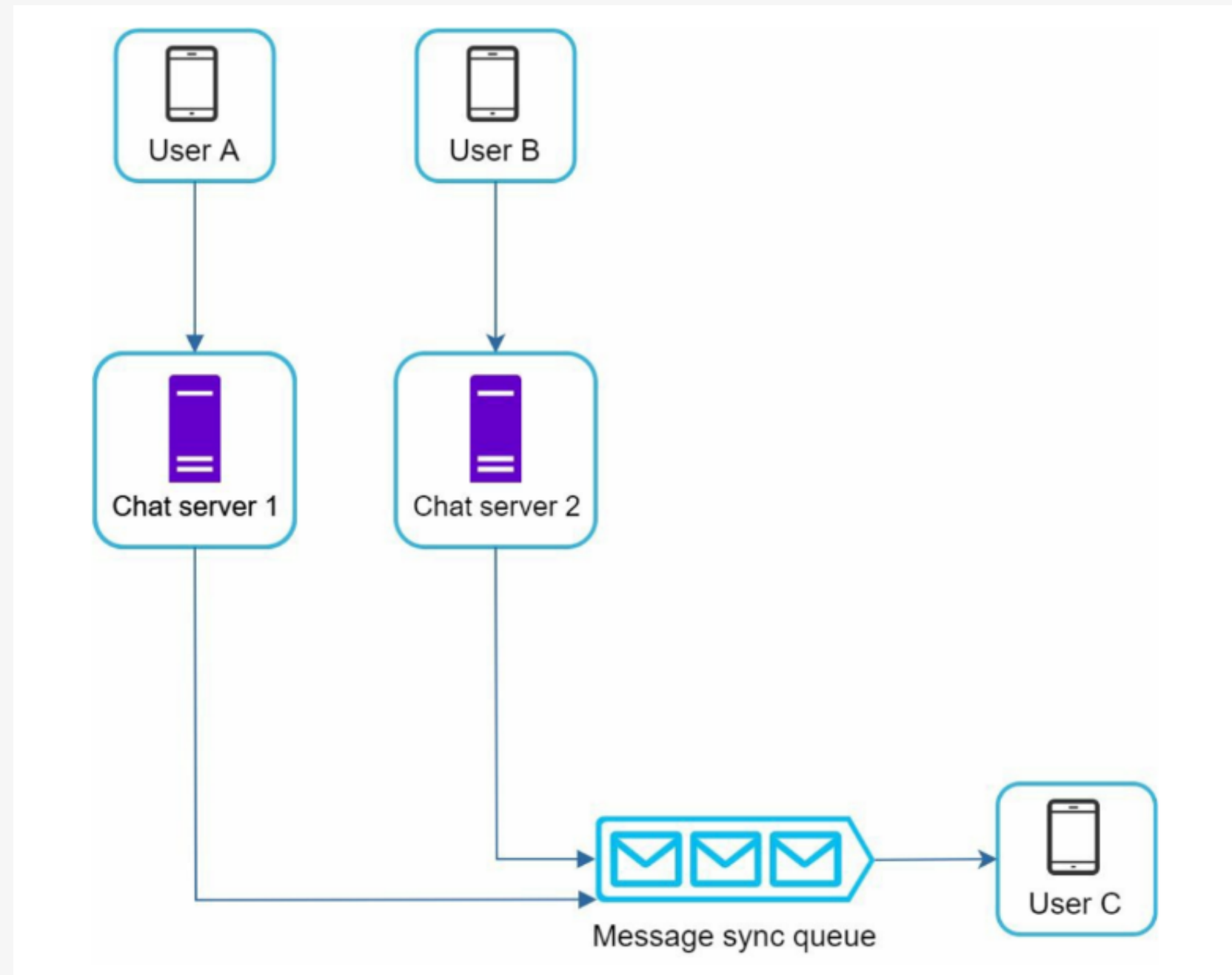
상세 설계 - 메시지 흐름

소규모 그룹채팅



상세 설계 - 메시지 흐름

소규모 그룹채팅 2



■ 상세 설계 - 메시지 흐름

대규모 그룹 처리

언급 안됨

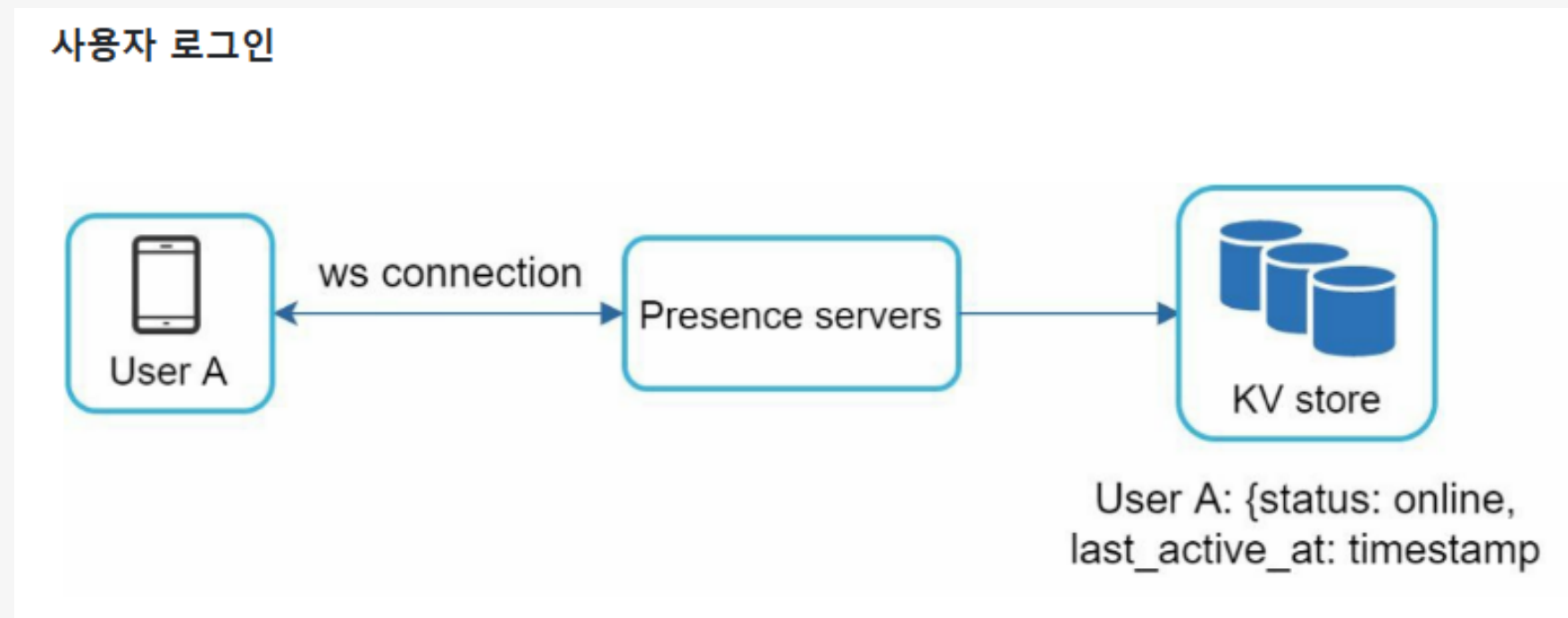
*일부 기술 블로그 기준

메시지 브로커를 써라

카프카를 써라 (2표)

상세 설계 - 접속 상태 확인

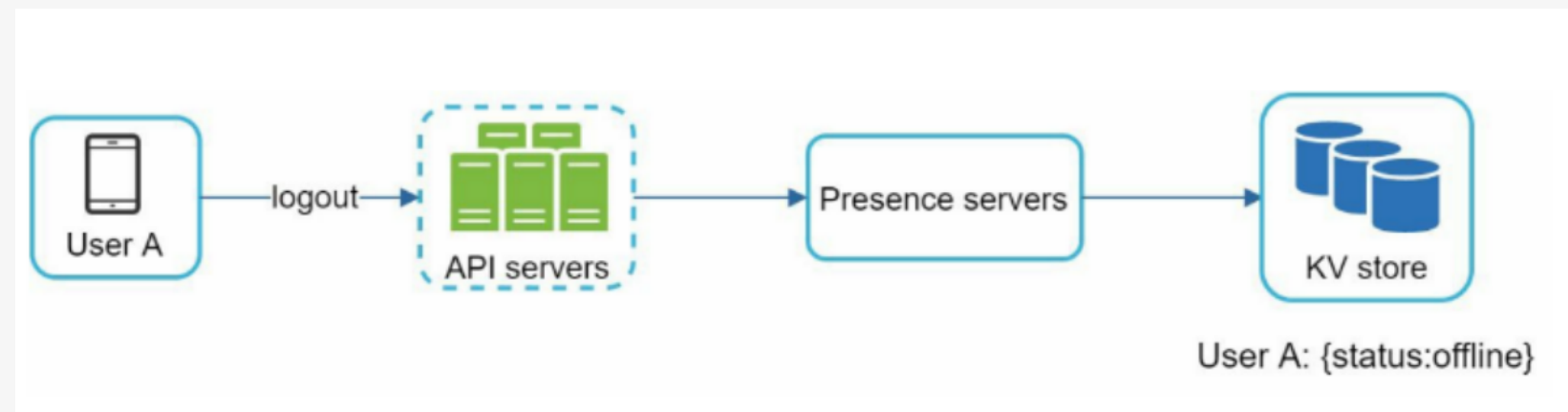
로그인



A의 상태와 last_active_at 타임스탬프 값을 키-값 저장소에 보관

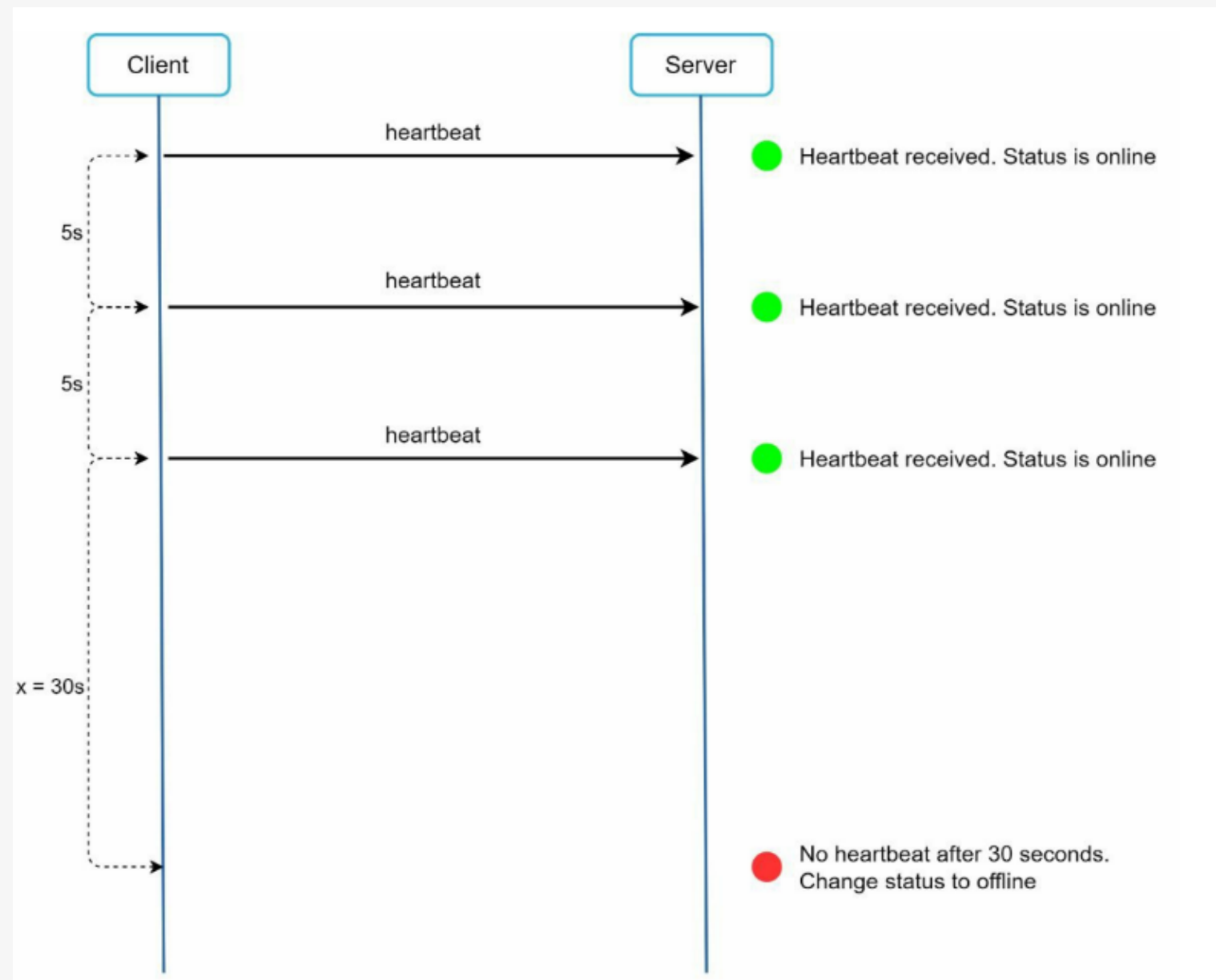
상세 설계 - 접속 상태 확인

로그아웃



상세 설계 - 접속 상태 확인

접속장애

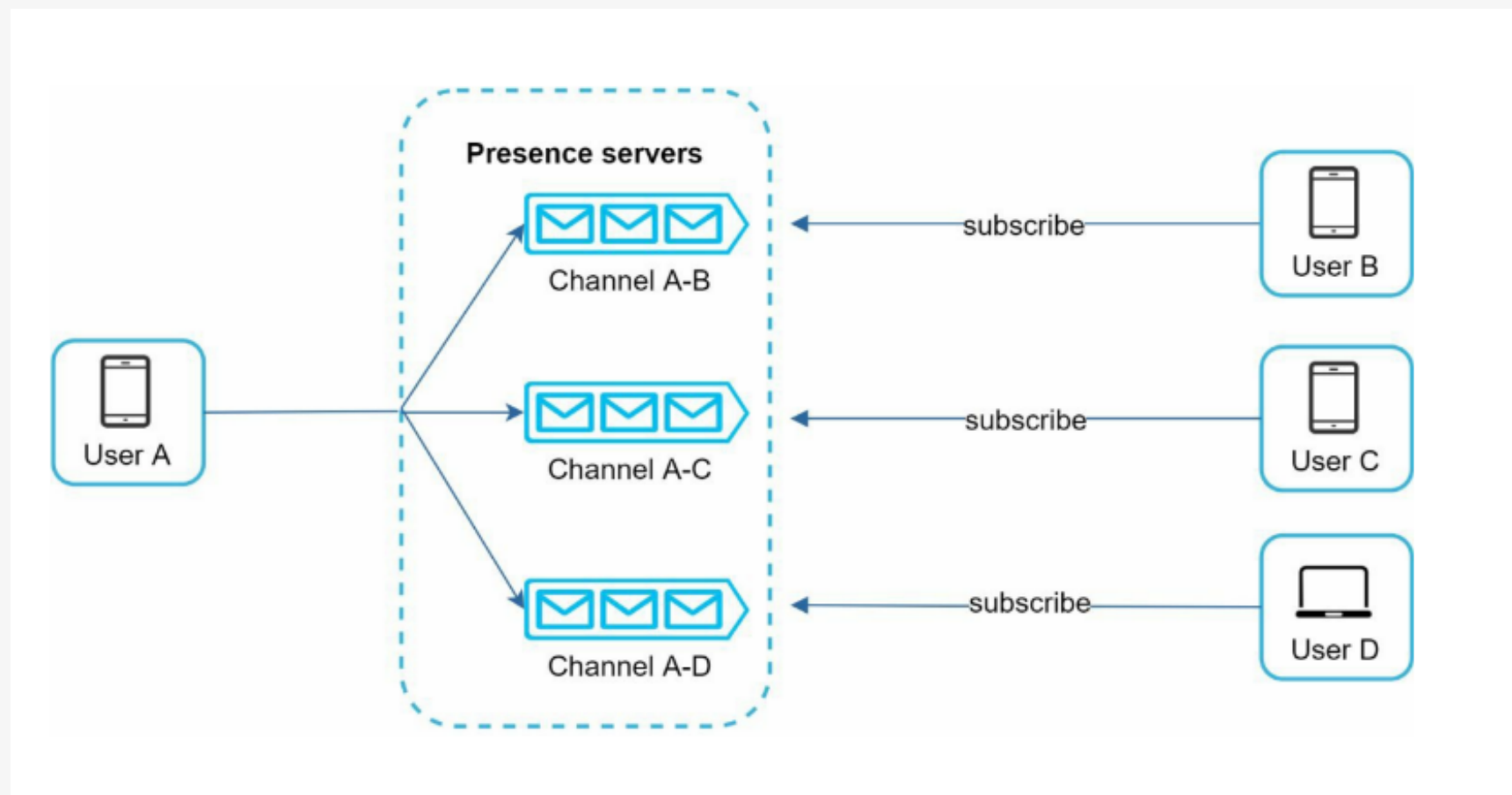


하트 비트 방식

N초간 응답 없으면 오프라인!

상세 설계 - 접속 상태 확인

수신자 측 정보 확인



발행-구독 모델(pub-sub model) 사용

사용자 A의 상태 변경

변경된 상태에 대한 메시지를 각 사용자별 구독 채널(A-B, A-C, A-D)에 전송
각 사용자 B,C,D는 자신의 구독 채널에서 상태변화 메시지를 통지받음

감사합니다